CSCI 4061 - Lab 09

11/09/2026

pine marten



UNIVERSITY OF MINNESOTA
Driven to Discover®

Overview

- TCP sockets
  - How TCP differs from the UDP
  - Connecting a client: getaddrinfo() -> socket() -> connect()
- HTTP 1.0
  - What a request and a response actually look like
- Lab: write a simple HTTP client that downloads three files

TCP vs. UDP

	UDP	TCP
Data	Separate datagrams	One continuous byte stream
Connection	None: every sendto() names the receiver	Set up once with connect(), then just read() / write()
Delivery	May be lost or reordered	Reliable and in order
Socket type	SOCK_DGRAM	SOCK_STREAM

- Once connected, a TCP socket behaves like a pipe: read() and write() on a file descriptor
- read() returning 0 means the other side closed the connection (end of stream), just like EOF on a file

Setting Up a TCP Client

1. getaddrinfo(): look up the server's address. Set up hints first:

```
struct addrinfo hints;
memset(&hints, 0, sizeof(hints));
hints.ai_family = AF_UNSPEC;     // IPv4 or IPv6
hints.ai_socktype = SOCK_STREAM; // TCP
int getaddrinfo(const char *host, const char *port,
    const struct addrinfo *hints, struct addrinfo **res);
```

2. socket(): use ai_family, ai_socktype, ai_protocol from the result
3. connect(): use ai_addr and ai_addrlen from the result
4. freeaddrinfo(): free the result on every path, success or error

HTTP 1.0

- Plain text: requests and responses are just strings sent over a TCP socket
- Every line ends in \r\n, not \n. An empty line (\r\n alone) ends the headers
- GET = retrieve a resource (a file) from the server

Request (client -> server)

```
GET /gatsby.txt HTTP/1.0 \r\n
\r\n
```

Response (server -> client)

```
HTTP/1.0 200 OK \r\n
Content-Type: text/plain \r\n
Content-Length: 1234 \r\n
\r\n
<file contents…>
```

Hints for lab

connect_to_server()

```
// set up hints (SOCK_STREAM)
// getaddrinfo(host, port, …)
//   error? use gai_strerror()
// sock_fd = socket(…)
// connect(sock_fd, …)
// freeaddrinfo(…)
// return sock_fd
```

write_http_request()

```
char buf[BUFSIZE];
// build the request string
//   in buf with snprintf()
// write() it to sock_fd
// return 0 or -1
```

- Don’t forget the leading “/” in the path and the final blank line
- Write strlen(buf) bytes, not BUFSIZE
- On an error after socket() succeeds, close() it and freeaddrinfo() before returning -1

Testing Your Client

- A working client creates three files in the lab directory:
  - index.html, gatsby.txt, ocelot.jpg
  - Open ocelot.jpg: if the image is corrupted, header skipping or your request is off
- make test-quiz: check your QUESTIONS.txt answers
- make test-code: run the automated code tests
- make test, then make zip and upload to Gradescope
  - Make sure the autograder matches your local results